

SELF STUDY DBT

1. Implement a trigger to update the product price in cart, when price is updated in products table.

```
kd3_98826_geetish> DELIMITER $$
kd3_98826_geetish>
kd3_98826_geetish> CREATE TRIGGER trig_price
-> AFTER UPDATE ON products
-> FOR EACH ROW
-> BEGIN
->     IF NEW.price != OLD.price THEN
->         UPDATE CART
->         SET price = NEW.price
->         WHERE product_id = NEW.id;
->     END IF;
-> END;
-> $$
Query OK, 0 rows affected (0.04 sec)
```

```
kd3_98826_geetish>
kd3_98826_geetish> DELIMITER ;
kd3_98826_geetish> INSERT INTO users (username, email) VALUES ('john_doe', 'john@example.com');
Query OK, 1 row affected (0.04 sec)

kd3_98826_geetish> INSERT INTO products (name, price, stock) VALUES ('Wireless Mouse', 50.00, 100);
Query OK, 1 row affected (0.02 sec)

kd3_98826_geetish> INSERT INTO cart (user_id, product_id, price, quantity) VALUES (1, 1, 50.00, 1);
Query OK, 1 row affected (0.03 sec)
```

```
kd3_98826_geetish> SELECT * FROM cart;
+-----+-----+-----+-----+-----+
| id | user_id | product_id | price | quantity |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 50.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

kd3_98826_geetish> UPDATE products SET price = 59.99 WHERE id = 1;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1 Changed: 1 Warnings: 0

kd3_98826_geetish> SELECT * FROM cart;
+-----+-----+-----+-----+-----+
| id | user_id | product_id | price | quantity |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 59.99 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

kd3_98826_geetish> -- Result: price will now automatically show 59.99!
kd3_98826_geetish> SELECT * FROM products;
+-----+-----+-----+-----+-----+
| id | name | price | stock |
+-----+-----+-----+-----+-----+
| 1 | Wireless Mouse | 59.99 | 100 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

2. Understand ON DELETE CASCADE, ON UPDATE CASCADE. Write emp, dept CREATE TABLE statements and apply CASCADE for all foreign keys.

MongoDb Lab Exercises:

1. Practice CRUD operations on Mongo.

1. Create two collections -- emp and dept.

```
company> db.dept.insertMany([
  { _id: 10, dname: "ACCOUNTING", loc: "NEW YORK" },
  { _id: 20, dname: "RESEARCH", loc: "DALLAS" },
  { _id: 30, dname: "SALES", loc: "CHICAGO" },
  { _id: 40, dname: "OPERATIONS", loc: "BOSTON" }
])
{
  acknowledged: true,
  insertedIds: { '0': 10, '1': 20, '2': 30, '3': 40 }
}
company> db.dept.find()
[
  { _id: 10, dname: 'ACCOUNTING', loc: 'NEW YORK' },
  { _id: 20, dname: 'RESEARCH', loc: 'DALLAS' },
  { _id: 30, dname: 'SALES', loc: 'CHICAGO' },
  { _id: 40, dname: 'OPERATIONS', loc: 'BOSTON' }
]
company> |
```

2. Insert the emp and dept records.

```
company> db.emp.insertMany([
  { _id: 7369, ename: "SMITH", job: "CLERK", mgr: 7902, hiredate: "1980-12-17", sal: 800, comm: 0, deptno: 20 },
  { _id: 7499, ename: "ALLEN", job: "SALESMAN", mgr: 7698, hiredate: "1981-02-20", sal: 1600, comm: 300, deptno: 30 },
  { _id: 7521, ename: "WARD", job: "SALESMAN", mgr: 7698, hiredate: "1981-02-22", sal: 1250, comm: 500, deptno: 30 },
  { _id: 7566, ename: "JONES", job: "MANAGER", mgr: 7839, hiredate: "1981-04-02", sal: 2975, comm: 0, deptno: 20 },
  { _id: 7654, ename: "MARTIN", job: "SALESMAN", mgr: 7698, hiredate: "1981-09-28", sal: 1250, comm: 1400, deptno: 30 },
  { _id: 7698, ename: "BLAKE", job: "MANAGER", mgr: 7839, hiredate: "1981-05-01", sal: 2850, comm: 0, deptno: 30 },
  { _id: 7782, ename: "CLARK", job: "MANAGER", mgr: 7839, hiredate: "1981-06-09", sal: 2450, comm: 0, deptno: 10 },
  { _id: 7788, ename: "SCOTT", job: "ANALYST", mgr: 7566, hiredate: "1987-04-19", sal: 3000, comm: 0, deptno: 20 },
  { _id: 7839, ename: "KING", job: "PRESIDENT", mgr: null, hiredate: "1981-11-17", sal: 5000, comm: 0, deptno: 10 },
  { _id: 7844, ename: "TURNER", job: "SALESMAN", mgr: 7698, hiredate: "1981-09-08", sal: 1500, comm: 0, deptno: 30 },
  { _id: 7876, ename: "ADAMS", job: "CLERK", mgr: 7788, hiredate: "1987-05-23", sal: 1100, comm: 0, deptno: 20 },
  { _id: 7900, ename: "JAMES", job: "CLERK", mgr: 7698, hiredate: "1981-12-03", sal: 950, comm: 0, deptno: 30 },
  { _id: 7902, ename: "FORD", job: "ANALYST", mgr: 7566, hiredate: "1981-12-03", sal: 3000, comm: 0, deptno: 20 },
  { _id: 7934, ename: "MILLER", job: "CLERK", mgr: 7782, hiredate: "1982-01-23", sal: 1300, comm: 0, deptno: 10 }
])
{
  acknowledged: true,
  insertedIds: {
    '0': 7369,
    '1': 7499,
    '2': 7521,
    '3': 7566,
    '4': 7654,
    '5': 7698,
    '6': 7782,
    '7': 7788,
    '8': 7839,
    '9': 7844,
    '10': 7876,
    '11': 7900,
    '12': 7902,
    '13': 7934
  }
}
```

3. Increase salary of dept 10 emps by 100.

```
company> db.emp.updateMany(
  { deptno: 10 },
  { $inc: { sal: 100 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
company> |
```

4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000.

```
company> { sal: { $gt: 1000 } }  
1000
```

```
company> db.emp.find(  
  { sal: { $gt: 1000 } },  
  {  
    _id: 0,  
    ename: 1,  
    job: 1,  
    deptno: 1,  
    sal: 1  
  }  
)  
.sort({  
  deptno: 1,  
  job: 1,  
  sal: -1  
)  
[  
  { ename: 'MILLER', job: 'CLERK', sal: 1400, deptno: 10 },  
  { ename: 'CLARK', job: 'MANAGER', sal: 2550, deptno: 10 },  
  { ename: 'KING', job: 'PRESIDENT', sal: 5100, deptno: 10 },  
  { ename: 'SCOTT', job: 'ANALYST', sal: 3000, deptno: 20 },  
  { ename: 'FORD', job: 'ANALYST', sal: 3000, deptno: 20 },  
  { ename: 'ADAMS', job: 'CLERK', sal: 1100, deptno: 20 },  
  { ename: 'JONES', job: 'MANAGER', sal: 2975, deptno: 20 },  
  { ename: 'BLAKE', job: 'MANAGER', sal: 2850, deptno: 30 },  
  { ename: 'ALLEN', job: 'SALESMAN', sal: 1600, deptno: 30 },  
  { ename: 'TURNER', job: 'SALESMAN', sal: 1500, deptno: 30 },  
  { ename: 'WARD', job: 'SALESMAN', sal: 1250, deptno: 30 },  
  { ename: 'MARTIN', job: 'SALESMAN', sal: 1250, deptno: 30 }  
]  
company> |
```

5. Delete the dept with id 40.

```
company> db.dept.deleteOne({ _id: 40 })  
{ acknowledged: true, deletedCount: 1 }  
company> |
```

2. Learn grouping operations using Mongo aggregation pipeline

1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT.

```
self_learning> db.emp.aggregate([
  {
    $match: {
      job: { $ne: "PRESIDENT" }
    }
  },
  {
    $group: {
      _id: "$job",
      totalSal: { $sum: "$sal" },
      maxSal: { $max: "$sal" },
      minSal: { $min: "$sal" },
      avgSal: { $avg: "$sal" }
    }
  }
])
[
  {
    _id: 'ANALYST',
    totalSal: 12000,
    maxSal: 3000,
    minSal: 3000,
    avgSal: 3000
  },
  {
    _id: 'MANAGER',
    totalSal: 16850,
    maxSal: 2975,
    minSal: 2550,
    avgSal: 2808.3333333333335
  },
  {
    _id: 'CLERK',
    totalSal: 8600,
    maxSal: 1500,
    minSal: 800,
    avgSal: 1075
  },
  {
    _id: 'SALESMAN',
    totalSal: 11200,
    maxSal: 1600,
    minSal: 1250,
    avgSal: 1400
  }
]
```

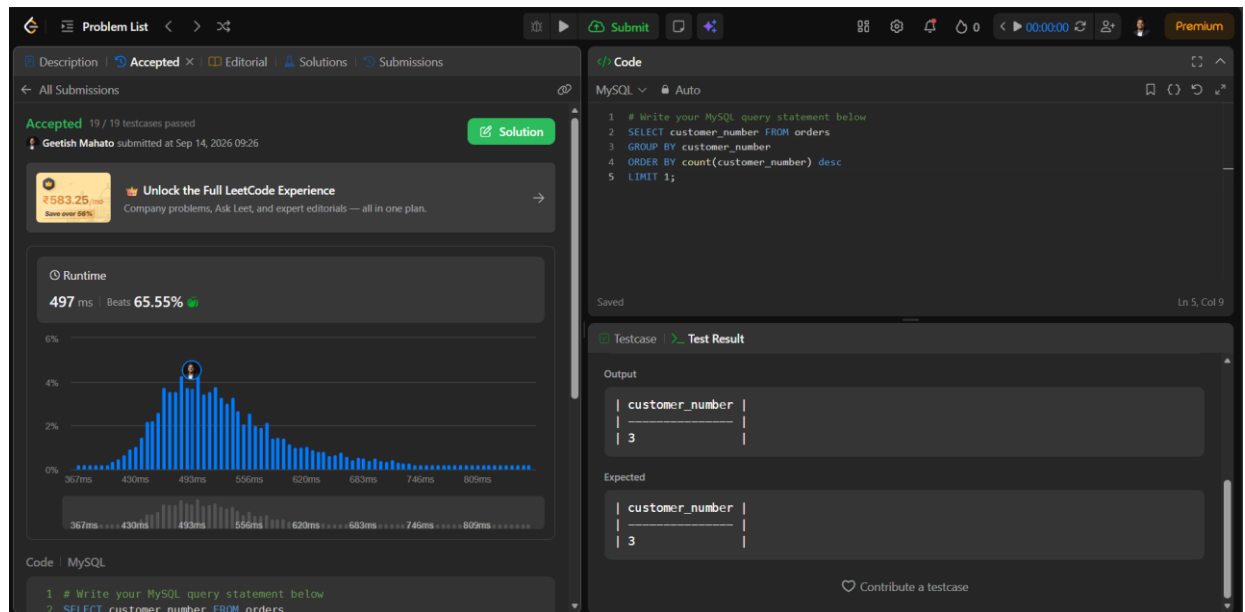
2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

```
self_learning> db.emp.aggregate([
  {
    $lookup: {
      from: "dept",
      localField: "deptno",
      foreignField: "deptno",
      as: "department"
    }
  },
  {
    $unwind: "$department"
  },
  {
    $project: {
      _id: 0,
      ename: 1,
      dname: "$department.dname",
      income: { $add: ["$sal", "$comm"] }
    }
  },
  {
    $sort: { income: 1 }
  }
])
[
  { ename: 'SMITH', dname: 'RESEARCH', income: null },
  { ename: 'SMITH', dname: 'RESEARCH', income: null },
  { ename: 'ALLEN', dname: 'SALES', income: null },
  { ename: 'ALLEN', dname: 'SALES', income: null },
  { ename: 'WARD', dname: 'SALES', income: null },
  { ename: 'WARD', dname: 'SALES', income: null },
  { ename: 'JONES', dname: 'RESEARCH', income: null },
  { ename: 'JONES', dname: 'RESEARCH', income: null },
  { ename: 'MARTIN', dname: 'SALES', income: null },
  { ename: 'MARTIN', dname: 'SALES', income: null },
  { ename: 'BLAKE', dname: 'SALES', income: null },
  { ename: 'BLAKE', dname: 'SALES', income: null },
  { ename: 'CLARK', dname: 'ACCOUNTING', income: null },
  { ename: 'CLARK', dname: 'ACCOUNTING', income: null },
  { ename: 'SCOTT', dname: 'RESEARCH', income: null },
  { ename: 'SCOTT', dname: 'RESEARCH', income: null },
  { ename: 'KING', dname: 'ACCOUNTING', income: null },
  { ename: 'KING', dname: 'ACCOUNTING', income: null },
  { ename: 'TURNER', dname: 'SALES', income: null },
  { ename: 'TURNER', dname: 'SALES', income: null }
]
```

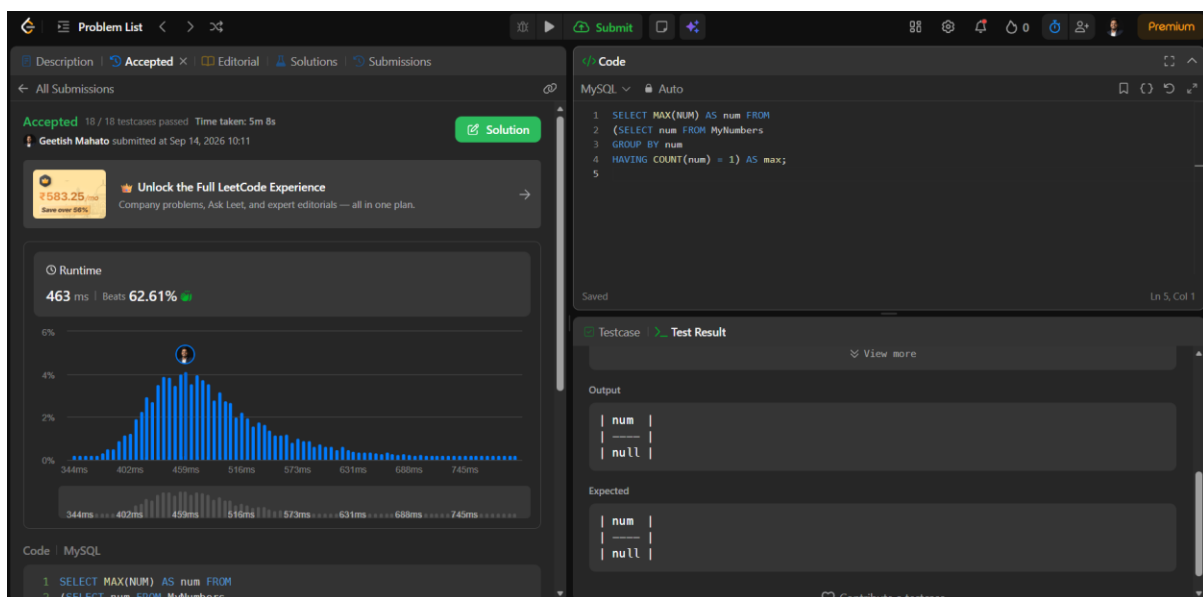
Leetcode Exercises

- GROUP BY Problems

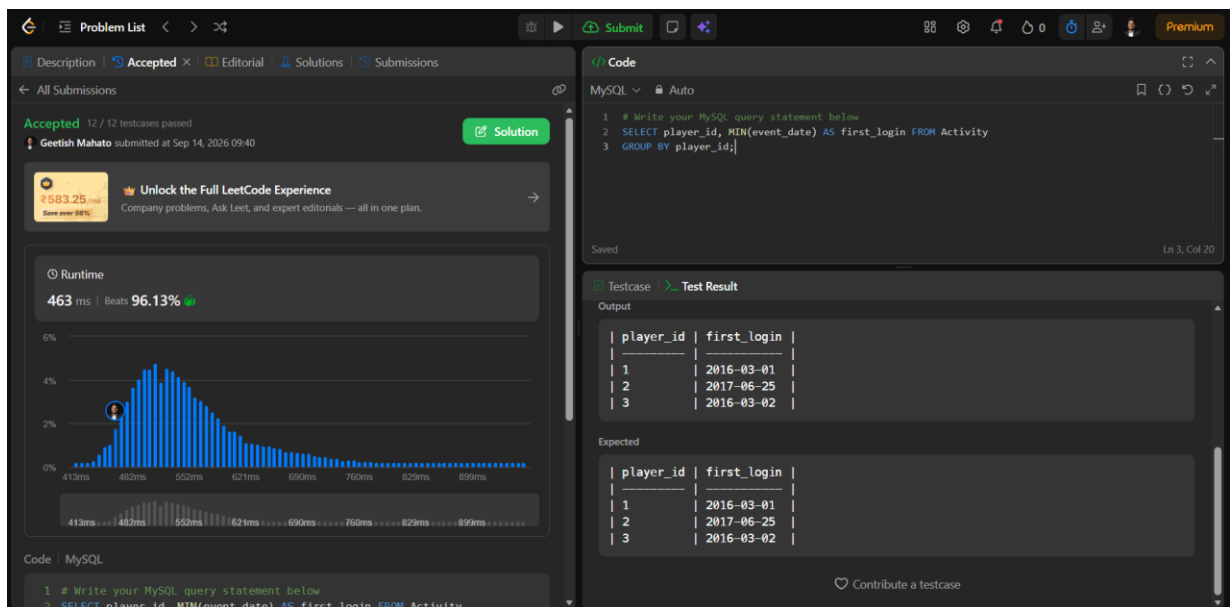
586. Customer Placing the Largest Number of Orders <https://leetcode.com/problems/customer-placing-the-largest-number-of-orders>



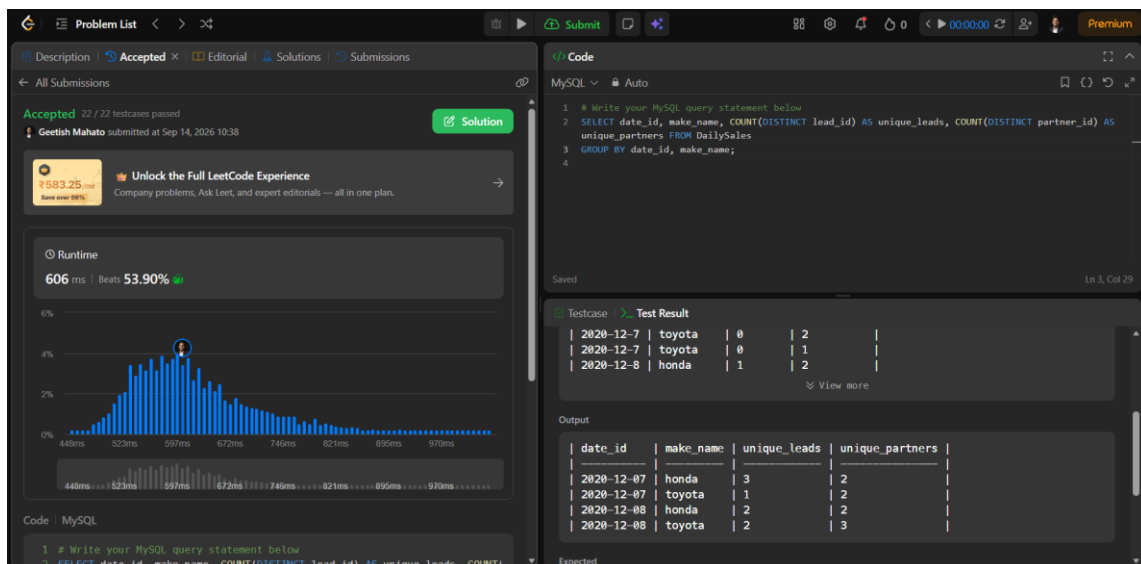
619. Biggest Single Number <https://leetcode.com/problems/biggest-single-number>



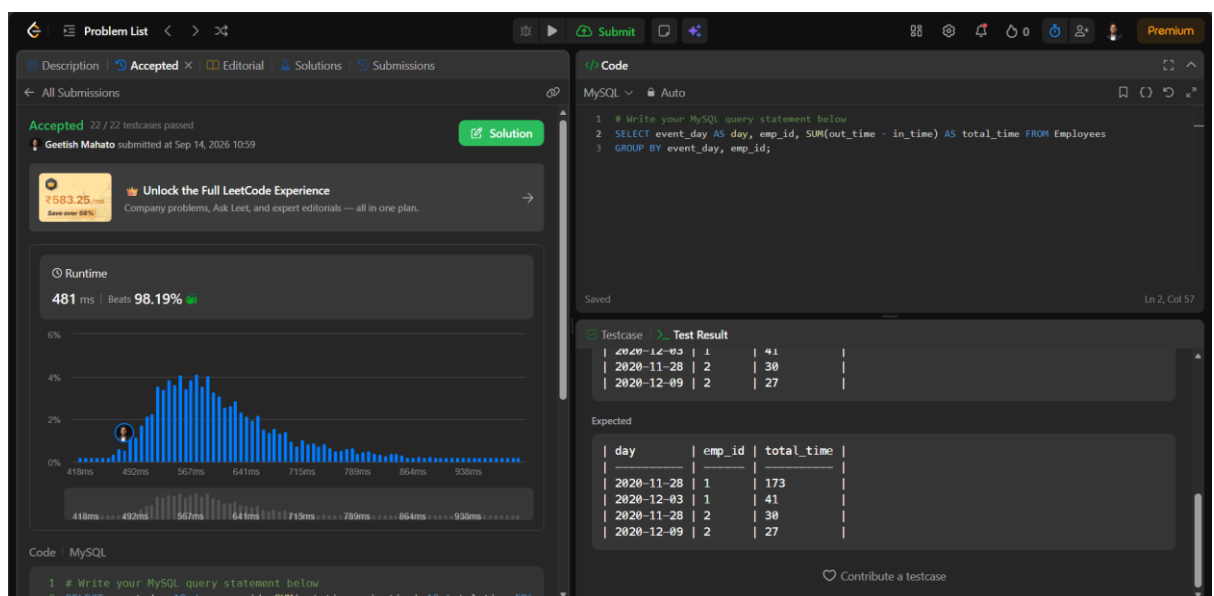
511. Game Play Analysis I <https://leetcode.com/problems/game-play-analysis-i>



1693. Daily Leads and Partners <https://leetcode.com/problems/daily-leads-and-partners>



1741. Find Total Time Spent by Each Employee <https://leetcode.com/problems/find-total-time-spent-by-each-employee>



1050. Actors and Directors Who Cooperated at Least Three Times

<https://leetcode.com/problems/actors-and-directors-who-cooperated-at-least-three-times>

Accepted 12 / 12 testcases passed
Geetish Mahato submitted at Sep 14, 2026 15:30

Runtime: 403 ms | Beats: 54.07%

Code: MySQL

```
1 # Write your MySQL query statement below
2 SELECT actor_id, director_id FROM ActorDirector
3 GROUP BY actor_id, director_id
4 HAVING COUNT(actor_id) >= 3;
```

Testcase: Test Result

Output

actor_id	director_id
1	1

Expected

actor_id	director_id
1	1

1890. The Latest Login in 2020 <https://leetcode.com/problems/the-latest-login-in-2020>

Example 1:

Input:

user_id	time_stamp
6	2020-06-30 15:06:07
6	2021-04-21 14:06:06
6	2019-03-07 00:18:15
8	2020-02-01 05:10:53
8	2020-12-30 00:46:50
2	2020-01-16 02:49:50
2	2019-08-25 07:59:08
14	2019-07-14 09:00:00
14	2021-01-06 11:59:59

Output:

user_id	last_stamp
6	2020-06-30 15:06:07
8	2020-12-30 00:46:50
2	2020-01-16 02:49:50

Explanation:

User 6 logged into their account 3 times but only once in 2020, so we include this login in the result table.

User 8 logged into their account 2 times in 2020, once in February and once in December. We include only the latest one (December) in the result table.

Testcase: Test Result

Accepted Runtime: 71 ms

Case 1

Input

user_id	time_stamp
6	2020-06-30 15:06:07
6	2021-04-21 14:06:06
6	2019-03-07 00:18:15
8	2020-02-01 05:10:53
8	2020-12-30 00:46:50
2	2020-01-16 02:49:50

1211. Queries Quality and Percentage <https://leetcode.com/problems/queries-quality-and-percentage>

Accepted 13 / 13 testcases passed
Geetish Mahato submitted at Sep 14, 2026 18:54

Runtime: 420 ms | Beats: 39.79%

Code: MySQL

```
1 # Write your MySQL query statement below
2 SELECT query_name, ROUND(AVG(rating/position), 2) AS quality, ROUND(SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END) / COUNT(*), 2) AS poor_query_percentage
3 FROM Queries
4 GROUP BY query_name;
```

Testcase: Test Result

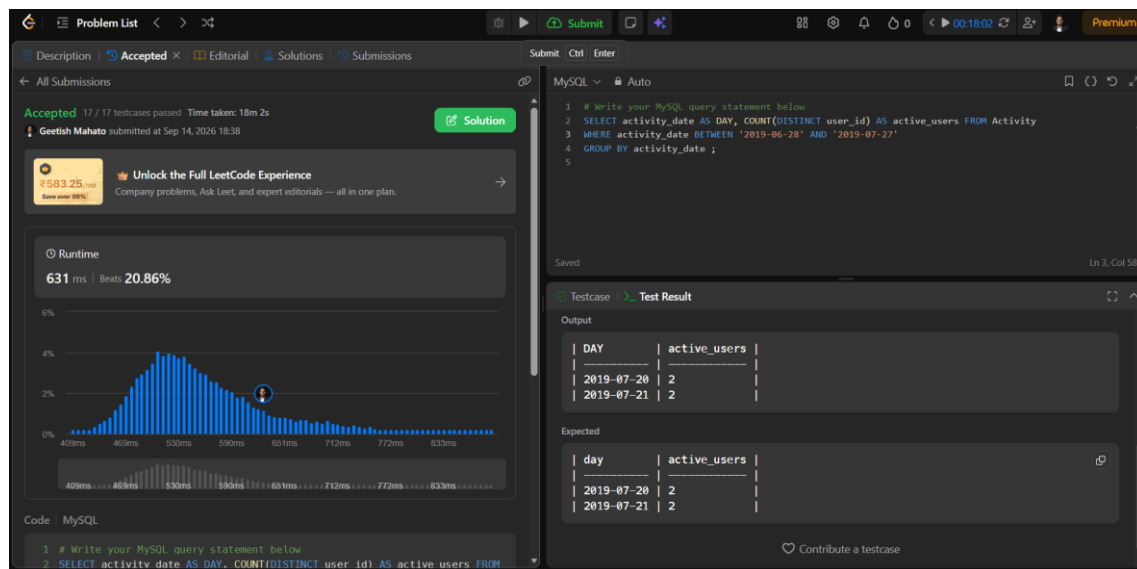
Output

query_name	quality	poor_query_percentage
Dog	2.5	33.33
Cat	0.66	33.33

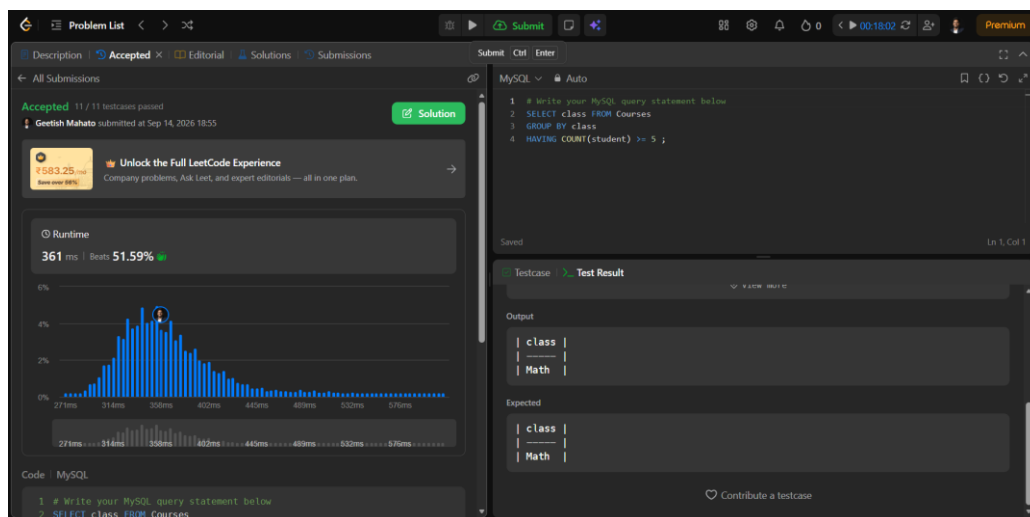
Expected

query_name	quality	poor_query_percentage
Dog	2.5	33.33
Cat	0.66	33.33

1141. User Activity for the Past 30 Days I <https://leetcode.com/problems/user-activity-for-the-past-30-days-i>

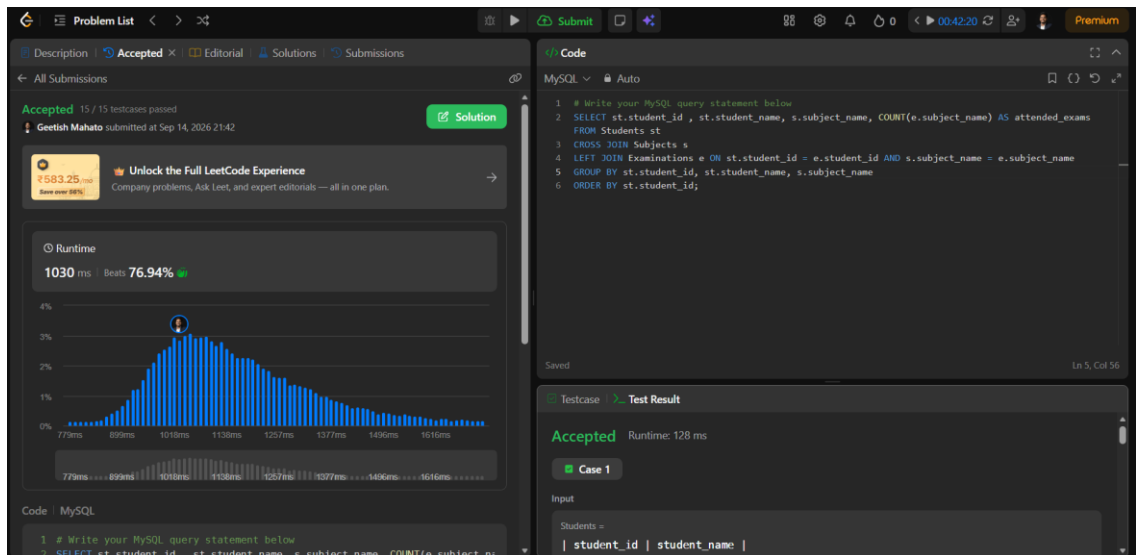


596. Classes With At Least 5 Students <https://leetcode.com/problems/classes-with-at-least-5-students>

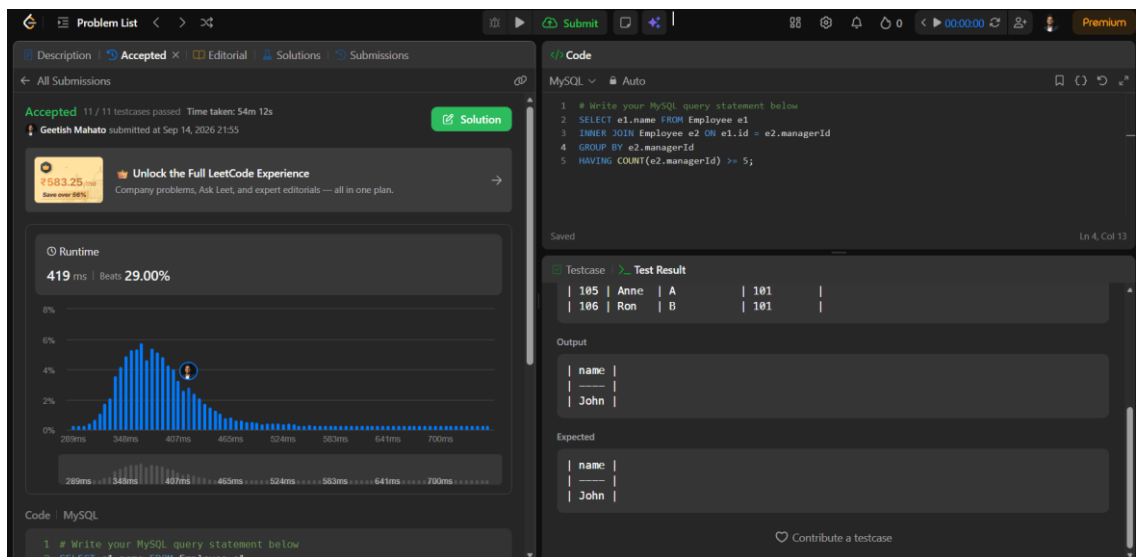


- JOINS Problems

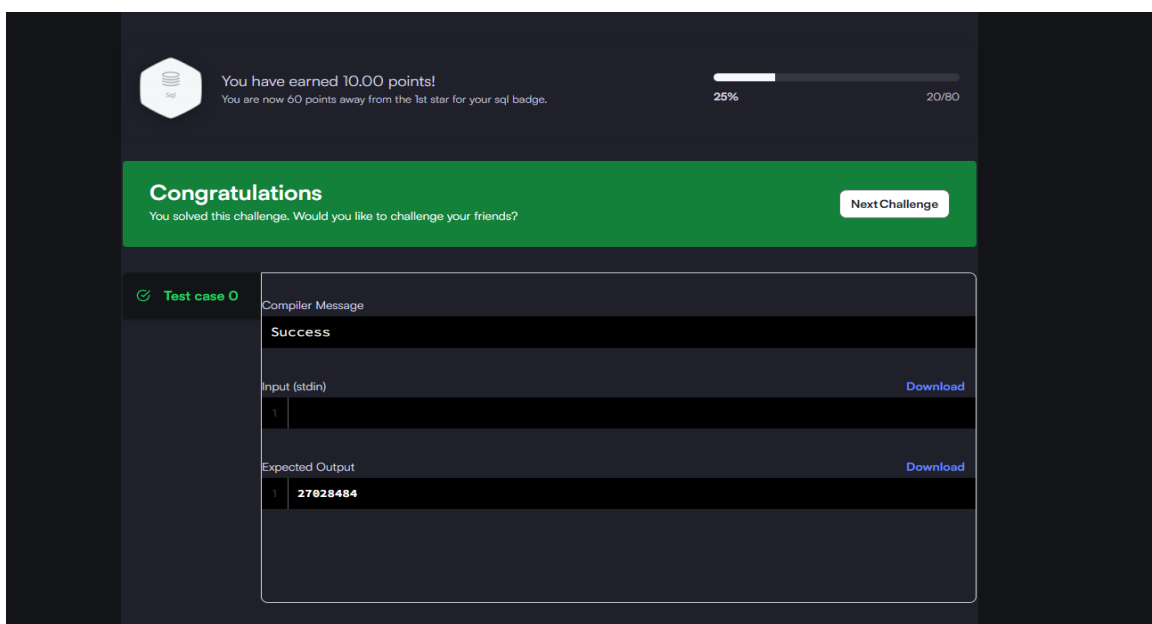
1. Replace Employee ID With The Unique Identifier <https://leetcode.com/problems/replace-employee-id-with-the-unique-identifier>



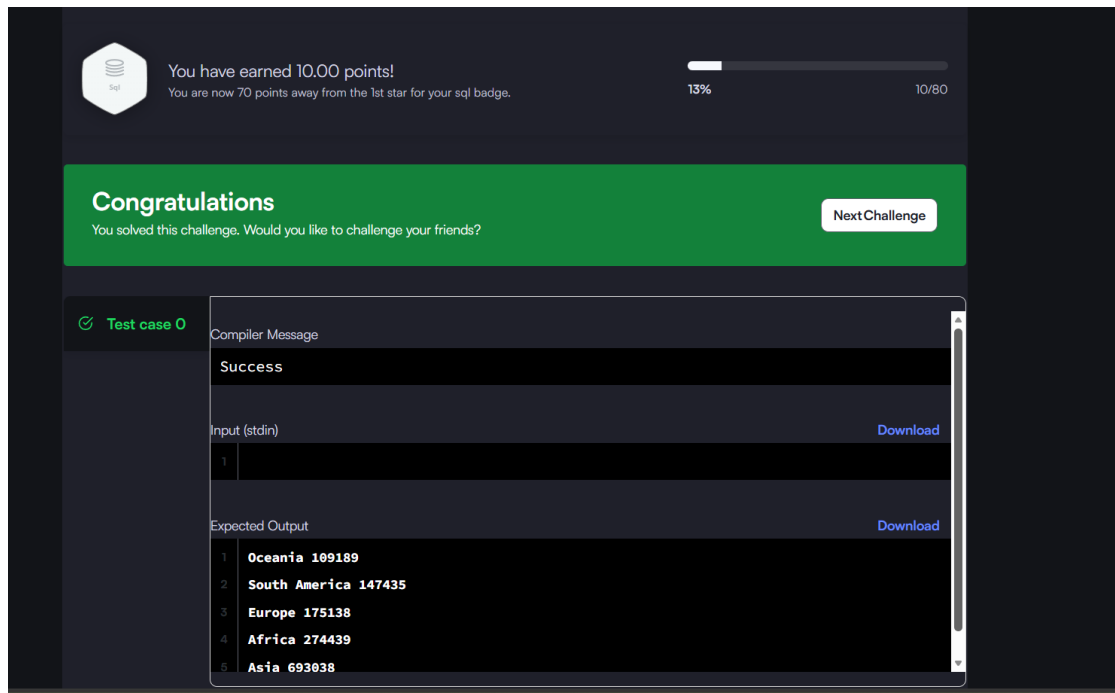
5. Managers With At Least 5 Direct Reports <https://leetcode.com/problems/managers-with-at-least-5-direct-reports>



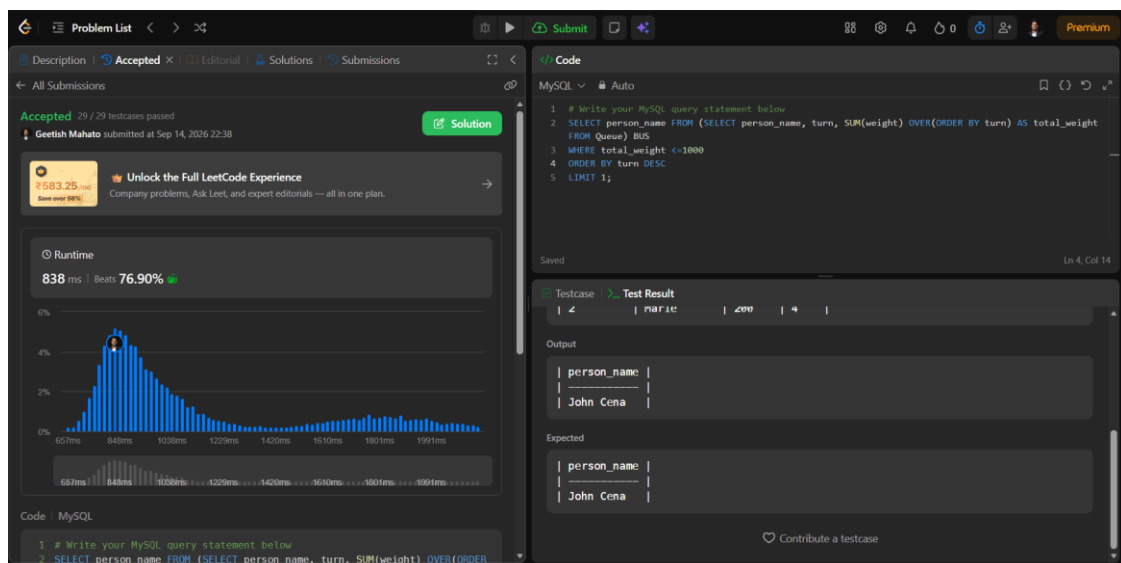
6. Asian Population <https://www.hackerrank.com/challenges/asian-population>



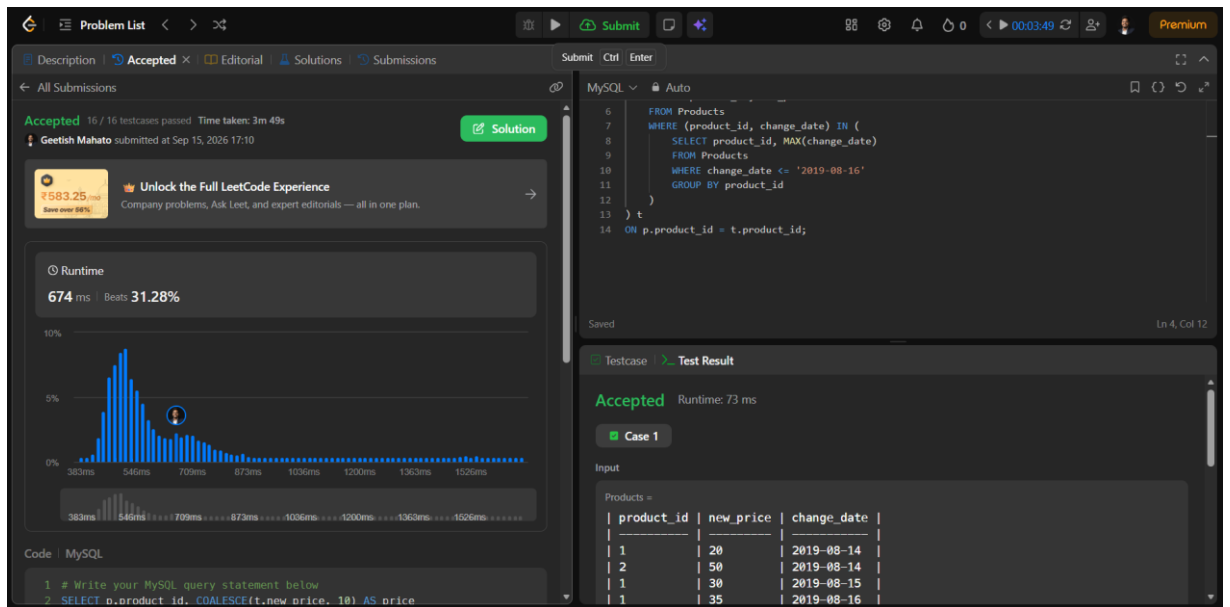
7. Average Population of Each Continent <https://www.hackerrank.com/challenges/average-population-of-each-continent>



8. Last Person to Fit in the Bus <https://leetcode.com/problems/last-person-to-fit-in-the-bus>

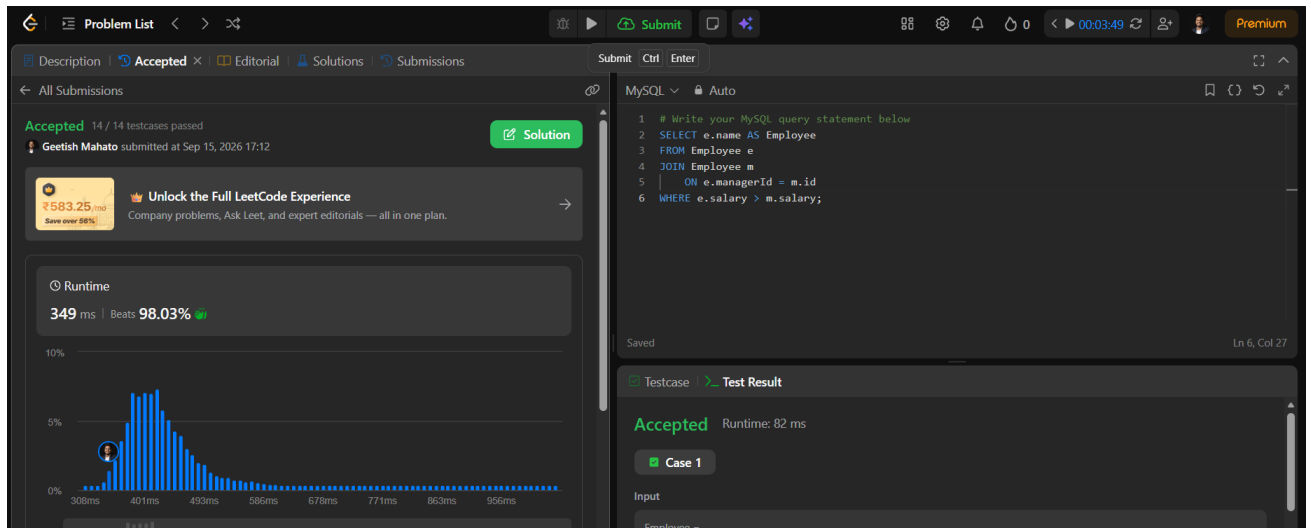


9. Product Price at a Given Date <https://leetcode.com/problems/product-price-at-a-given-date>

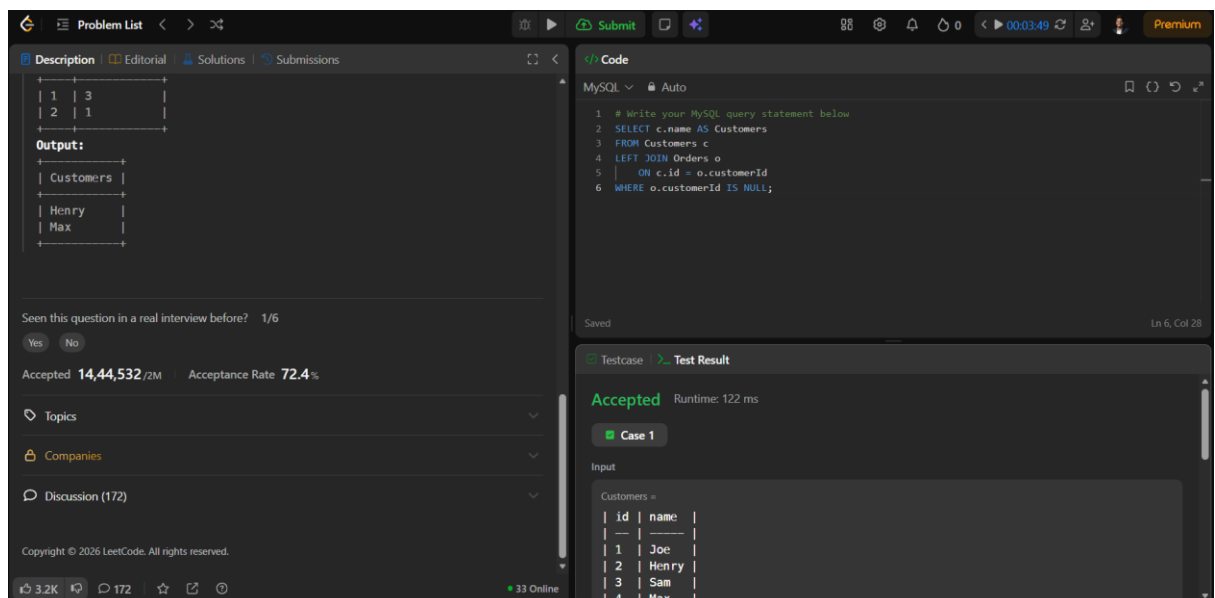


- Subquery Problems

181. Employees Earning More Than Their Managers <https://leetcode.com/problems/employees-earning-more-than-their-managers/>



183. Customers Who Never Order <https://leetcode.com/problems/customers-who-never-order/>



197. Rising Temperature <https://leetcode.com/problems/rising-temperature/>

Accepted 15 / 15 testcases passed
Geetish Mahato submitted at Sep 15, 2026 17:15

Runtime
483 ms | Beats 75.01%

Code MySQL

```
1 SELECT w1.id
2 FROM Weather w1
3 JOIN Weather w2
4   ON w1.recordDate = DATE_ADD(w2.recordDate, INTERVAL 1 DAY)
5 WHERE w1.temperature > w2.temperature;
```

Testcase **Test Result**
Accepted Runtime: 65 ms

Case 1

Input

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

607. Sales Person <https://leetcode.com/problems/sales-person/>

Accepted 21 / 21 testcases passed
Geetish Mahato submitted at Sep 15, 2026 17:17

Runtime
1636 ms | Beats 48.11%

Code MySQL

```
1 # Write your MySQL query statement below
2 SELECT s.name
3 FROM SalesPerson s
4 WHERE NOT EXISTS (
5   SELECT 1
6   FROM Orders o
7   JOIN Company c
8     ON o.com_id = c.com_id
9   WHERE o.sales_id = s.sales_id
10    AND c.name = 'RED'
11 );
```

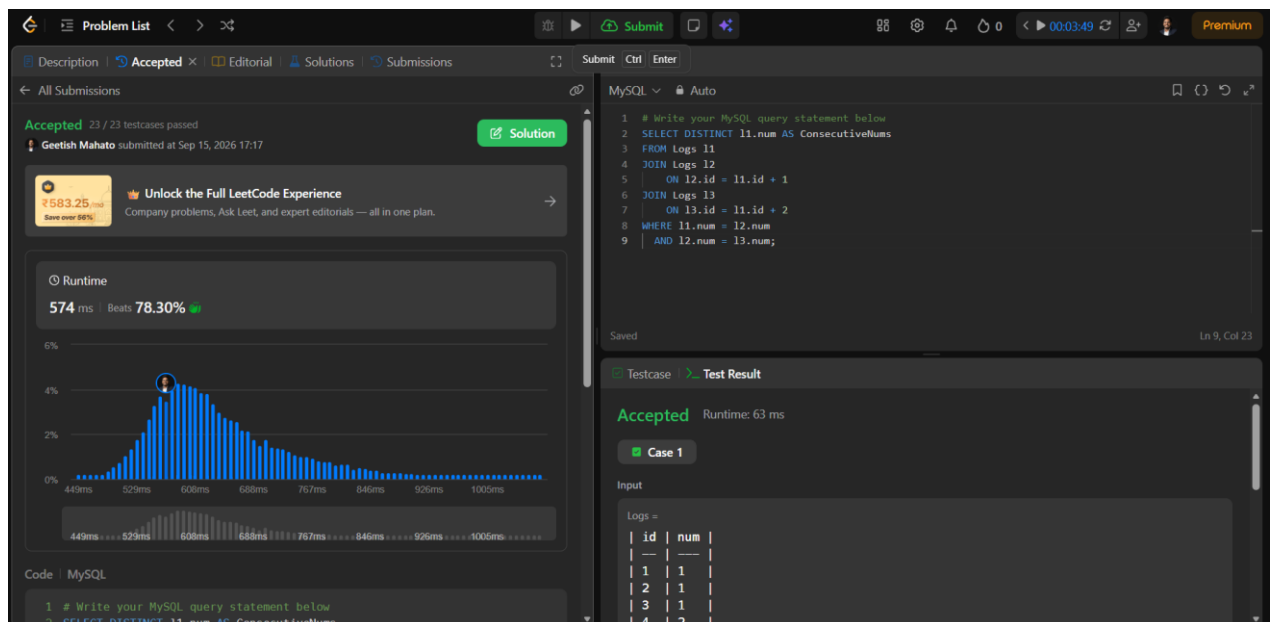
Testcase **Test Result**
Accepted Runtime: 146 ms

Case 1

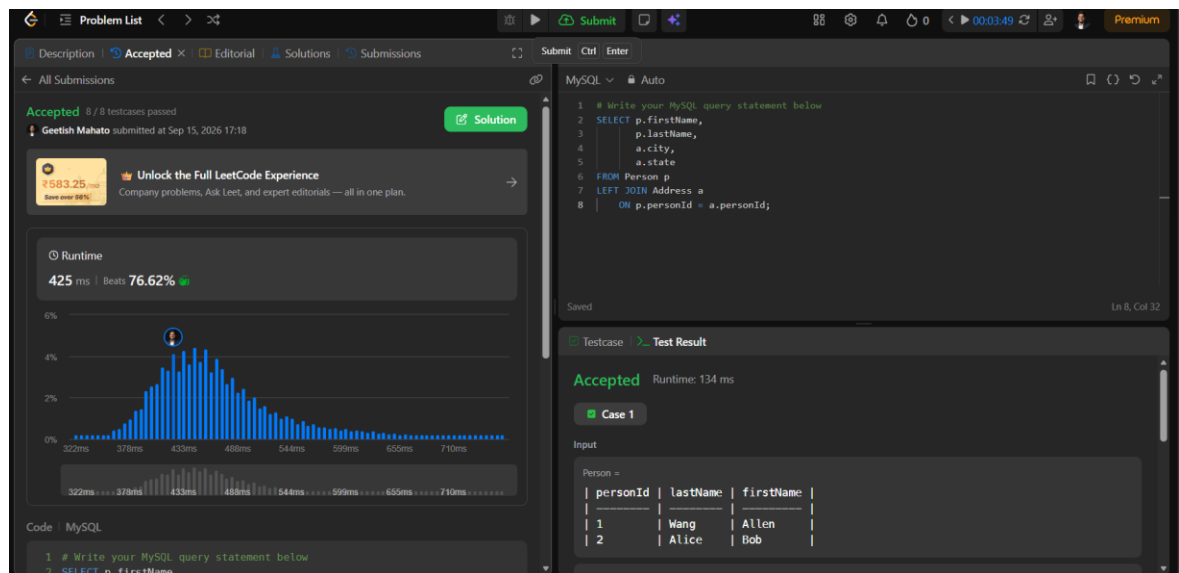
Input

sales_id	name	salary	commission_rate	hire_date
1	John	100000	6	4/1/2006
2	Amy	12000	5	5/1/2010
3	Mark	65000	12	12/25/2008
4	Pam	25000	25	1/1/2005

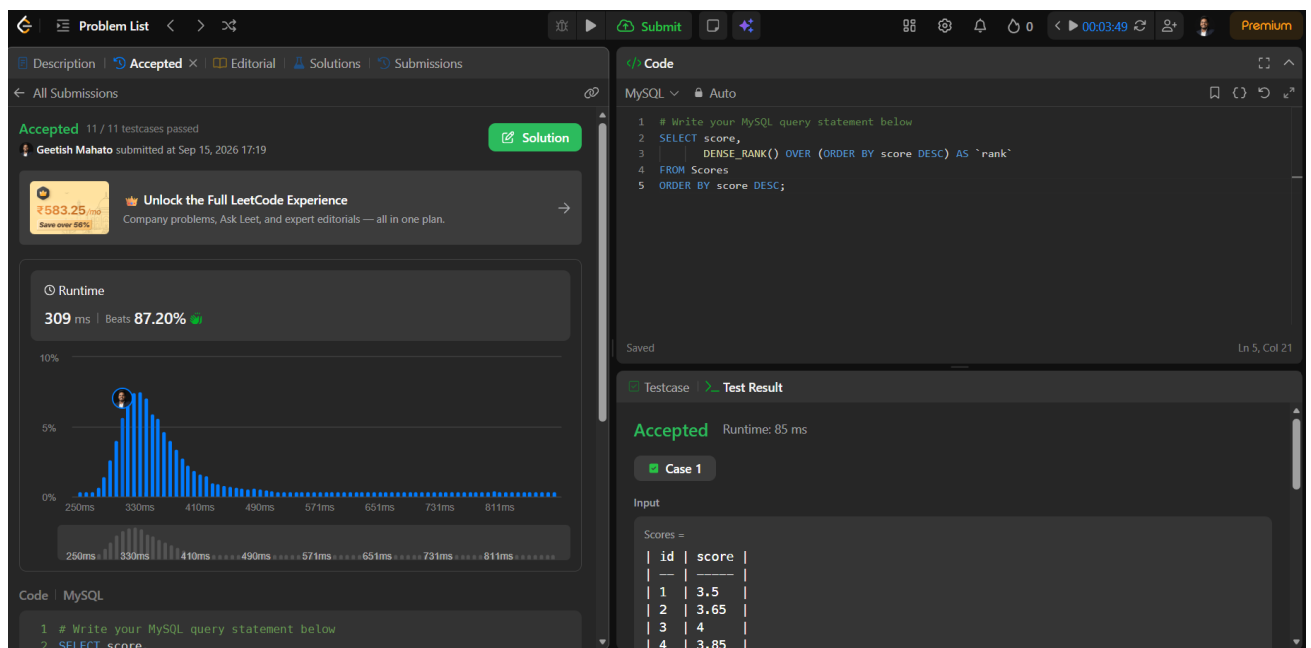
180. Consecutive Numbers <https://leetcode.com/problems/consecutive-numbers/>



175. Combine Two Tables <https://leetcode.com/problems/combine-two-tables/>



178. Rank Scores <https://leetcode.com/problems/rank-scores/>



570. Managers with at Least 5 Direct Reports <https://leetcode.com/problems/managers-with-at-least-5-direct-reports>

The screenshot displays the LeetCode interface for problem 570. The left sidebar shows the problem status as 'Accepted' with 11/11 testcases passed. The runtime is 374 ms, beating 61.76% of submissions. The MySQL query solution is shown in the code editor:

```
1 # Write your MySQL query statement below
2 SELECT e1.name FROM Employee e1
3 INNER JOIN Employee e2 ON e1.id = e2.managerId
4 GROUP BY e2.managerId
5 HAVING COUNT(e2.managerId) >= 5;
```

The test result section shows 'Accepted' with a runtime of 69 ms for Case 1. The input data is as follows:

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101

196. Delete Duplicate Emails <https://leetcode.com/problems/delete-duplicate-emails/>

The screenshot displays the LeetCode interface for problem 196. The left sidebar shows the problem status as 'Accepted' with 22/22 testcases passed. The runtime is 1097 ms, beating 16.32% of submissions. The MySQL query solution is shown in the code editor:

```
1 # Write your MySQL query statement below
2 DELETE p1
3 FROM Person p1
4 JOIN Person p2
5 ON p1.email = p2.email
6 AND p1.id > p2.id;
```

The test result section shows 'Accepted' with a runtime of 106 ms for Case 1. The input data is as follows:

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

1204. Last Person to Fit in the Bus <https://leetcode.com/problems/last-person-to-fit-in-the-bus/>

Accepted 29 / 29 testcases passed
Geetish Mahato submitted at Sep 15, 2026 17:22

Runtime
881 ms | Beats 65.90%

Code MySQL

```
1 # Write your MySQL query statement below
2 SELECT person_name
3 FROM (
4     SELECT person_name,
5            turn,
6            SUM(weight) OVER (ORDER BY turn) AS total_weight
7     FROM Queue
8 ) q
9 WHERE total_weight <= 1000
10 ORDER BY turn DESC
11 LIMIT 1;
```

Testcase **Test Result**
Accepted Runtime: 82 ms

Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3

176. Second Highest Salary <https://leetcode.com/problems/second-highest-salary>

Accepted 10 / 10 testcases passed
Geetish Mahato submitted at Sep 15, 2026 17:24

Runtime
309 ms | Beats 48.16%

Code MySQL

```
1 # Write your MySQL query statement below
2 SELECT MAX(salary) AS SecondHighestSalary
3 FROM Employee
4 WHERE salary < (
5     SELECT MAX(salary)
6     FROM Employee
7 );
```

Testcase **Test Result**
Accepted Runtime: 113 ms

Case 1 **Case 2**

Input

Employee =

id	salary
1	100
2	200
3	300

CDAC Admission Process Case Study

- Design database for CDAC Admission System.
 - Create appropriate tables and insert sample records.

```
kd3_98826_geetish> SHOW TABLES;
+-----+
| Tables_in_cdac_admission |
+-----+
| allocations               |
| centers                   |
| course_centers            |
| courses                   |
| preferences               |
| students                  |
+-----+
6 rows in set (0.00 sec)
```

```
kd3_98826_geetish> SELECT * FROM centers;
+-----+-----+-----+
| center_id | center_name | city |
+-----+-----+-----+
|          1 | Sunbeam Pune | Pune |
|          2 | Sunbeam Mumbai | Mumbai |
|          3 | CDAC Bangalore | Bangalore |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
kd3_98826_geetish> INSERT INTO students(student_name, email, merit_rank,
category)
-> VALUES
-> ('Rahul', 'rahul@gmail.com', 1, 'OPEN'),
-> ('Priya', 'priya@gmail.com', 2, 'OPEN'),
-> ('Amit', 'amit@gmail.com', 3, 'OPEN'),
-> ('Sneha', 'sneha@gmail.com', 4, 'OPEN'),
-> ('Rohit', 'rohit@gmail.com', 5, 'OPEN'),
-> ('Neha', 'neha@gmail.com', 6, 'OPEN');
Query OK, 6 rows affected (0.00 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

- Write a Stored Procedure for Round1 allocation.

```

kd3_98826_geetish> DELIMITER $
kd3_98826_geetish> CREATE PROCEDURE round1_allocation(
-> OUT p_allocated INT
-> )
-> BEGIN
->
-> DECLARE v_done INT DEFAULT 0;
-> DECLARE v_student_id INT;
-> DECLARE v_course_center_id INT;
-> DECLARE v_seats INT;
-> DECLARE v_allocated_seats INT;
->
-> DECLARE cur_students CURSOR FOR
-> SELECT student_id
-> FROM students
-> ORDER BY merit_rank;
->
-> DECLARE CONTINUE HANDLER FOR NOT FOUND
-> SET v_done = 1;
->
-> DECLARE EXIT HANDLER FOR SQLEXCEPTION
-> BEGIN
-> ROLLBACK;
-> SET p_allocated = -1;
-> END;
->
-> SET p_allocated = 0;
->
-> START TRANSACTION;
->
-> OPEN cur_students;
->
-> student_loop: LOOP
->
-> FETCH cur_students INTO v_student_id;
->
-> IF v_done = 1 THEN
-> LEAVE student_loop;
-> END IF;
->
-> SET v_course_center_id = NULL;
->
-> SELECT course_center_id
-> INTO v_course_center_id
-> FROM preferences
-> WHERE student_id = v_student_id
-> AND preference_no = 1;
->
-> IF v_course_center_id IS NOT NULL THEN
->
-> SELECT total_seats
-> INTO v_seats
-> FROM course_centers
-> WHERE course_center_id = v_course_center_id;
->
-> SELECT COUNT(*)
-> INTO v_allocated_seats
-> FROM allocations
-> WHERE course_center_id = v_course_center_id;
->
-> IF v_allocated_seats < v_seats THEN
->
-> INSERT INTO allocations(
-> student_id,
-> course_center_id,
-> round_no
-> )
-> VALUES(
-> v_student_id,
-> v_course_center_id,
-> 1
-> );
->
-> SET p_allocated = p_allocated + 1;
->
-> END IF;
->
-> END IF;
->
-> END LOOP;
->
-> CLOSE cur_students;
->
-> COMMIT;
->
-> END $
Query OK, 0 rows affected (0.02 sec)

kd3_98826_geetish> DELIMITER ;
kd3_98826_geetish>

```

```

kd3_98826_geetish> CALL round1_allocation(@count);
Query OK, 0 rows affected (0.02 sec)

```

```
kd3_98826_geetish> SELECT @count;
```

```

+-----+
| @count |
+-----+
|      4 |
+-----+
1 row in set (0.00 sec)

```

```
kd3_98826_geetish> SELECT * FROM allocations;
```

```

+-----+-----+-----+-----+-----+
| allocation_id | student_id | course_center_id | round_no | allocated_at |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 1 | 2026-09-15 17:36:07 |
| 2 | 2 | 1 | 1 | 2026-09-15 17:36:07 |
| 3 | 5 | 2 | 1 | 2026-09-15 17:36:07 |
| 4 | 6 | 2 | 1 | 2026-09-15 17:36:07 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

- Write a Stored Procedure for Round2 allocation.

```

kd3_98826_geetish> DELIMITER $
kd3_98826_geetish> CREATE PROCEDURE round2_allocation(
->   OUT p_allocated INT
-> )
-> BEGIN
->
->   DECLARE v_done INT DEFAULT 0;
->   DECLARE v_student_id INT;
->   DECLARE v_course_center_id INT;
->   DECLARE v_seats INT;
->   DECLARE v_allocated_seats INT;
->
->   DECLARE cur_students CURSOR FOR
->     SELECT s.student_id
->     FROM students s
->     LEFT JOIN allocations a
->       ON s.student_id = a.student_id
->     WHERE a.student_id IS NULL
->     ORDER BY s.merit_rank;
->
->   DECLARE CONTINUE HANDLER FOR NOT FOUND
->     SET v_done = 1;
->
->   DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->       ROLLBACK;
->       SET p_allocated = -1;
->     END;
->
->   SET p_allocated = 0;
->
->   START TRANSACTION;
->
->   OPEN cur_students;
->
->   student_loop: LOOP
->
->     FETCH cur_students INTO v_student_id;
->
->     IF v_done = 1 THEN
->       LEAVE student_loop;
->     END IF;
->
->     SET v_course_center_id = NULL;
->
->     SELECT course_center_id
->     INTO v_course_center_id
->     FROM preferences
->     WHERE student_id = v_student_id
->     AND preference_no = 2;
->
->     IF v_course_center_id IS NOT NULL THEN
->
->       SELECT total_seats
->       INTO v_seats
->       FROM course_centers
->       WHERE course_center_id = v_course_center_id;
->
->       SELECT COUNT(*)
->       INTO v_allocated_seats
->       FROM allocations
->       WHERE course_center_id = v_course_center_id;
->
->       IF v_allocated_seats < v_seats THEN
->
->         INSERT INTO allocations(
->           student_id,
->           course_center_id,
->           round_no
->         )
->         VALUES(
->           v_student_id,
->           v_course_center_id,
->           2
->         );
->
->         SET p_allocated = p_allocated + 1;
->
->       END IF;
->
->     END IF;
->
->   END LOOP;
->
->   CLOSE cur_students;
->
->   COMMIT;
->
-> END $
Query OK, 0 rows affected (0.01 sec)

```

```

kd3_98826_geetish> SELECT
->   s.student_id,
->   s.student_name,
->   s.merit_rank,
->   c.course_name,
->   ce.center_name,
->   a.round_no
-> FROM allocations a
-> JOIN students s
->   ON a.student_id = s.student_id
-> JOIN course_centers cc
->   ON a.course_center_id = cc.course_center_id
-> JOIN courses c
->   ON cc.course_id = c.course_id
-> JOIN centers ce
->   ON cc.center_id = ce.center_id
-> ORDER BY s.merit_rank;

```

student_id	student_name	merit_rank	course_name	center_name	round_no
1	Rahul	1	DAC	Sunbeam Pune	1
2	Priya	2	DAC	Sunbeam Pune	1
3	Amit	3	DBDA	Sunbeam Pune	2
4	Sneha	4	DBDA	Sunbeam Pune	2
5	Rohit	5	DAC	Sunbeam Mumbai	1
6	Neha	6	DAC	Sunbeam Mumbai	1

6 rows in set (0.01 sec)